KAPASA MAKASA UNIVERSITY

School of Science, Information communication Technology

CYS 121: INFORMATION SYSTEMS SECURITY

Laboratory Manual – Unit 2: Access Control

Student Name	Incognito
Student Number	222222222
Course Code	CYS 121 – Information Systems Security
Unit Title	Unit 2: Access Control
Lecturer	Dr. Lungu Nelson
Submission Date	unkown
Platform	Ubuntu (WSL2) on Windows

IT SOCIETY KMU – presented by the IT SOCIETY Project coordinator

1. Introduction

Access control is one of the foundational pillars of information systems security. It defines who can access which resources, under what conditions, and with what level of privilege. This laboratory exercise applies core access control concepts to a simulated banking environment running on a local Ubuntu system.

The lab implements Discretionary Access Control (DAC) using Linux file permission bits, then extends the model using Access Control Lists (ACLs) to handle cross-role access requirements that the standard owner-group-other model cannot express. The exercise concludes with a least-privilege test that verifies both permitted and prohibited actions.

1.1 Learning Objectives Addressed

- Explain the relationship between subjects, objects, and access rights in a real operating system.
- Apply Linux ownership and permission bits to enforce discretionary access control.
- Use groups and ACLs to simulate role-based access control in a banking scenario.
- Test least privilege by proving that permitted actions succeed and prohibited actions fail.

2. Environment Setup and Preparation

All activities were performed exclusively on a local Ubuntu environment (WSL2 on Windows). No external systems, real customer data, or production services were accessed at any point during this exercise.

2.1 Working Directory Creation

The working directory was created using the following commands:

```
mkdir -p ~/CYS121_Labs/unit2_access_control/bank_case
cd ~/CYS121_Labs/unit2_access_control/bank_case
```

The -p flag ensures that all parent directories are created in a single operation, avoiding errors if intermediate directories do not yet exist. The bank_case folder serves as the root for all files and subdirectories created in this lab.

2.2 ACL Package Installation

The acl package provides the setfacl and getfacl commands, which are essential for Part C of this lab. Installation was performed with:

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo apt install -y acl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
```

The -y flag automatically confirms the installation prompt. After installation, the availability of setfacl and getfacl was verified by running each command with the --version flag.

2.3 System Identification

System identification details were recorded to document the exact machine that produced the evidence:

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# date
Tue Apr  7 18:36:17 CAT 2026
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# hostname
DESKTOP-K7CKH77
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# hostname -I
172.25.99.242 172.17.0.1
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

```
date        # Records date and time of the session
hostname    # Records the machine name
hostname -I # Records the IP address assigned to the machine
```

Screenshot evidence: u2_setup.png – Shows the working directory, ACL installation output, and system identification details.

3. Part A – Building Users, Groups, and Protected Folders

3.1 Theoretical Background

In a discretionary access control model, access decisions are based on the identity of the subject (user or process) and the ownership of the object (file or directory). Linux implements DAC through three layers: owner, group, and other. By grouping users into roles first, we move toward a role-based approach where permissions attach to roles rather than individuals.

In this banking scenario, three business roles are modelled. A teller handles daily transaction records. A supervisor oversees and can review teller work. An auditor inspects all folders for compliance but never modifies records. Each role maps directly to a Linux group, ensuring clean separation of privilege.

3.2 Commands Executed

Step 1 – Create Role Groups

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo groupadd teller
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo groupadd supervisor
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo groupadd auditor
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# cat /etc/group | grep -E "teller|supe
rvisor|auditor"
teller:x:1001:
supervisor:x:1002:
auditor:x:1003:
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

Each groupadd command creates an entry in /etc/group. Using separate groups for each business role means that permissions can be granted at the role level rather than to individuals, making the system easier to maintain and audit.

Step 2 – Create User Accounts

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo adduser teller2
info: Adding user `teller2' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group `teller2' (1004) ...
info: Adding new user `teller2' (1004) with group `teller2 (1004)' ...
info: Creating home directory `/home/teller2' ...
info: Copying files from `/etc/skel' ...
New password:
Retype new password:
Sorry, passwords do not match.
passwd: Authentication token manipulation error
passwd: password unchanged
Try again? [y/N] y
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for teller2
Enter the new value, or press ENTER for the default
        Full Name []:
        Room Number []:
        Work Phone []:
        Home Phone []:
        Other []:
Is the information correct? [Y/n] y
info: Adding new user `teller2' to supplemental / extra groups `users' ...
```

```
sudo adduser teller2
sudo adduser super2
sudo adduser audit2
```

Simple temporary passwords were used because the focus of this exercise is authorisation, not authentication strength. In a production system, strong passwords and multi-factor authentication would be mandatory.

Step 3 – Assign Users to Groups

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo usermod -aG teller teller2
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo usermod -aG supervisor super2
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo usermod -aG auditor audit2
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

The -aG flags append the user to the specified group without removing them from any existing groups. This is safer than using -G alone, which would overwrite all existing group memberships.

Step 4 – Verify Group Membership

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# id teller2
uid=1004(teller2) gid=1004(teller2) groups=1004(teller2),100(users),1001(teller)
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# id super2
uid=1005(super2) gid=1005(super2) groups=1005(super2),100(users),1002(supervisor)
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# id audit2
uid=1006(audit2) gid=1006(audit2) groups=1006(audit2),100(users),1003(auditor)
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

```
id teller2
id super2
id audit2
```

The id command displays the user ID (UID), primary group ID (GID), and all supplementary groups. Verifying membership before applying permissions prevents misconfiguration errors.

Step 5 – Create Folder Structure and Sample Files

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# mkdir transactions approvals auditlog
s
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# touch transactions/daybook.txt approv
als/approvals.txt auditlogs/audit.txt
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# echo "Daily teller summary" > transac
tions/daybook.txt
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# echo "Approval records" > approvals/a
pprovals.txt
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# echo "Audit log entries" > auditlogs/
audit.txt
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

Each folder represents a protected resource associated with one business role. Adding sample text content makes the access tests in Part D meaningful because actual data can be read or denied.

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# ls -la
total 20
drwxr-xr-x 5 root root 4096 Apr  7 19:45 .
drwxr-xr-x 3 root root 4096 Apr  7 18:29 ..
drwxr-xr-x 2 root root 4096 Apr  7 19:45 approvals
drwxr-xr-x 2 root root 4096 Apr  7 19:45 auditlogs
drwxr-xr-x 2 root root 4096 Apr  7 19:45 transactions
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# ls -la transactions/
total 12
drwxr-xr-x 2 root root 4096 Apr  7 19:45 .
drwxr-xr-x 5 root root 4096 Apr  7 19:45 ..
-rw-r--r-- 1 root root   21 Apr  7 19:46 daybook.txt
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

4. Part B – Discretionary Access Control with chmod and chown

4.1 Theoretical Background

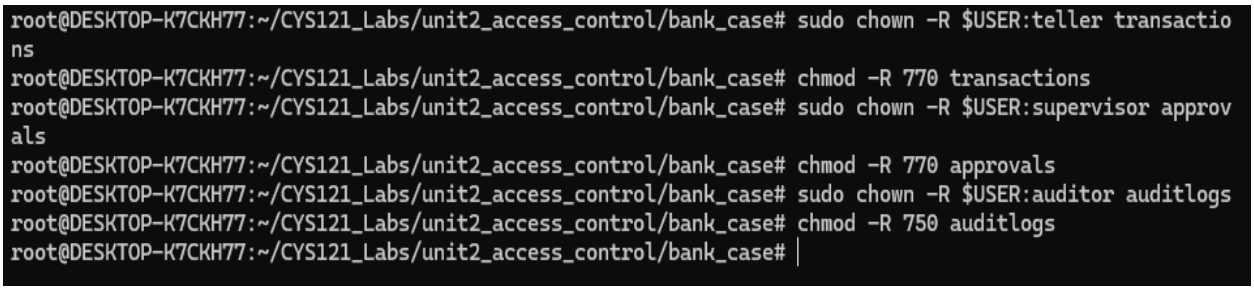
Linux permission bits are the primary mechanism for discretionary access control. Every file and directory has three sets of read (r), write (w), and execute (x) bits: one set for the **owner**, one for the **group**, and one for **everyone else** (other). In octal notation, read = 4, write = 2, and execute = 1. These values are summed to produce the permission digit for each category.

For a directory, the execute bit has special meaning: it controls the ability to enter the directory (traverse it). A user without execute permission on a directory cannot list its contents or access files within it, even if they have read permission on the files themselves.

4.2 Permission Values Explained

Folder	Owner	Group	Permission (octal)	Others
transactions	$USER (rwx)	teller (rwx)	770	No access
approvals	$USER (rwx)	supervisor (rwx)	770	No access
auditlogs	$USER (rwx)	auditor (r-x)	750	No access

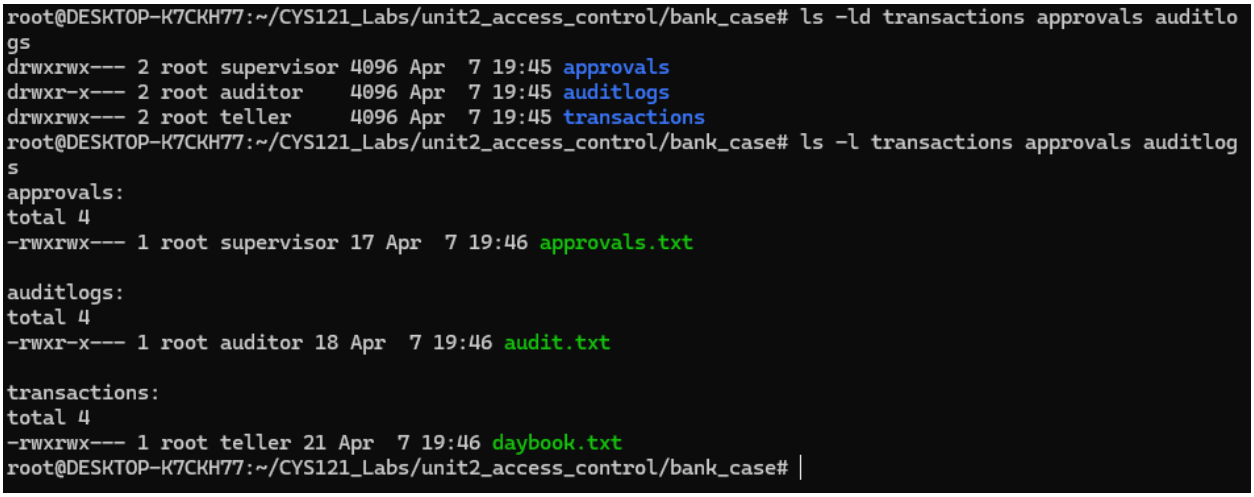
4.3 Commands Executed




```
sudo chown -R $USER:teller transactions
chmod -R 770 transactions
sudo chown -R $USER:supervisor approvals
chmod -R 770 approvals
sudo chown -R $USER:auditor auditlogs
chmod -R 750 auditlogs
```

The -R flag applies the change recursively to all files and subdirectories. Using $USER as the file owner preserves administrative control of all three directories under the student account, while the group assignment ensures that the appropriate role can access each space.

4.4 Verification




```
ls -ld transactions approvals auditlogs
ls -l transactions approvals auditlogs
```

The ls -ld command displays the permissions, owner, and group of each directory itself (rather than its contents). A typical output line for the transactions folder would appear as:

```
drwxrwx--- 2 student teller 4096 [date] transactions
```

Reading left to right: d indicates a directory; rwx is the owner (full access); rwx is the group (full access); --- is others (no access). The owner field shows the student account and the group field shows teller, confirming the chown command succeeded.

5. Part C – Extending the Access Model with Access Control Lists

5.1 Theoretical Background

The standard owner-group-other model has an inherent limitation: each file or directory has exactly one group assignment. When a business rule requires a user from a different group to have access (for example, an auditor who must read the transactions folder owned by the teller group), the only options under the basic model would be to add the auditor to the teller group (breaking role separation) or to change the permissions for “others” (granting access to all users on the system). Both options are security risks.

Access Control Lists (ACLs) solve this problem by attaching additional user-specific or group-specific rules to any file or directory, layered on top of the existing DAC model. The Linux kernel evaluates ACL entries before the “other” permission class, enabling precise cross-role access without compromising role boundaries.

5.2 Commands Executed

Auditor read access to transactions

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# setfacl -m u:audit2:rx transactions
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# setfacl -m u:audit2:r-- transactions/
daybook.txt
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

```
setfacl -m u:audit2:rx transactions
setfacl -m u:audit2:r-- transactions/daybook.txt
```

Auditor read access to approvals

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# setfacl -m u:audit2:rx approvals
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# setfacl -m u:audit2:r-- approvals/app
rovals.txt
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

Supervisor read access to transactions

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# setfacl -m u:super2:rx transactions
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# setfacl -m u:super2:r-- transactions/
daybook.txt
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

The -m flag modifies the ACL. The entry format is u:username:permissions, where u specifies a user entry, the username identifies the account, and the permissions are expressed as symbolic bits. Granting rx on a directory allows entry and listing; granting r-- on a file allows reading without modification.

Display ACL Entries

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# getfacl transactions approvals auditl
ogs
# file: transactions
# owner: root
# group: teller
user::rwx
user:super2:r-x
user:audit2:r-x
group::rwx
mask::rwx
other::---

# file: approvals
# owner: root
# group: supervisor
user::rwx
user:audit2:r-x
group::rwx
mask::rwx
other::---

# file: auditlogs
# owner: root
# group: auditor
user::rwx
group::r-x
other::---
```

The mask entry is automatically created by the kernel when ACLs are applied. It represents the maximum effective permissions that any named user or group ACL entry can have. As long as the mask is not restricted, named user entries operate at the level specified.

5.3 Why ACLs Are Valuable for Cross-Role Access

In this banking scenario, the auditor must inspect transaction records as part of a compliance requirement, yet must not belong to the teller group and must not be able to write to those records. The ACL system satisfies all three constraints simultaneously. The auditor account receives a user-specific read and execute entry on the transactions directory, while the group owner (teller) retains full read-write-execute rights and others retain no rights. This granularity is impossible to achieve with chmod alone.

A second example is the supervisor's read access to transactions. A supervisor reviews teller work for approval but should not edit daybook entries directly. The ACL entry grants exactly read and execute on the folder and read-only on the file, matching the business rule without altering any group membership or the broader permission model.

6. Part D – Testing Least Privilege

6.1 Theoretical Background

The principle of least privilege states that every subject should be granted only the minimum access rights necessary to perform its authorised functions, and nothing more. Testing this principle requires two types of evidence: a successful permitted action (proving that legitimate access works) and a failed prohibited action (proving that the access control mechanism actually blocks unauthorised operations).

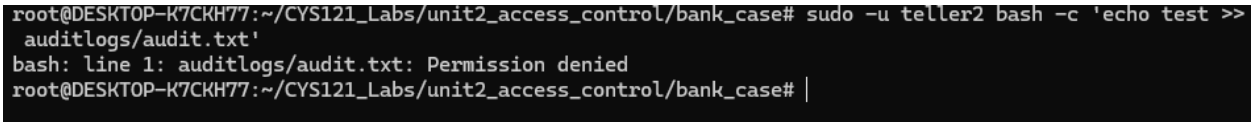
The sudo -u command allows us to impersonate a user account without logging out of the administrative session. This technique is appropriate for lab testing because it avoids the overhead of switching sessions while still executing commands under the target account's effective permissions.

6.2 Commands Executed and Expected Results

Permitted actions (expected to succeed), we test Least Privilege and these tests are expected to be successful access files

```
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo -u teller2 cat transactions/dayb
ook.txt
Daily teller summary
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo -u super2 cat approvals/approval
s.txt
Approval records
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo -u audit2 cat auditlogs/audit.tx
t
Audit log entries
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo -u audit2 cat transactions/daybo
ok.txt
Daily teller summary
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo -u audit2 cat approvals/approval
s.txt
Approval records
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case# sudo -u super2 cat transactions/daybo
ok.txt
Daily teller summary
root@DESKTOP-K7CKH77:~/CYS121_Labs/unit2_access_control/bank_case#
```

Prohibited actions (expected to fail with Permission denied)




```
sudo -u teller2 bash -c 'echo test >> approvals/approvals.txt'
sudo -u audit2 bash -c 'echo test >> transactions/daybook.txt'
```

When a prohibited action is attempted, the kernel returns an error because the subject does not have write permission and no ACL entry grants write to that account. The terminal displays a message like on the above screenshort.

This message is itself security evidence. It confirms that the operating system enforced the access control decision and did not allow the write operation to proceed.

6.3 Access Matrix

The following table summarises the effective access rights for each role after both the chmod/chown configuration (Part B) and the ACL extensions (Part C) were applied. Results are based on the actual test outcomes observed in Part D.

User / Role	transactions/	approvals/	auditlogs/	Source of Access
teller2 (teller)	READ/WRITE/EXEC	**NO ACCESS**	**NO ACCESS**	Group ownership
super2 (supervisor)	READ/EXEC	READ/WRITE/EXEC	**NO ACCESS**	ACL (transactions); Group (approvals)
audit2 (auditor)	READ/EXEC	READ/EXEC	READ/EXEC	ACL (transactions, approvals); Group (auditlogs)

Legend: READ/WRITE/EXEC = full access; READ/EXEC = read and traverse, no write; NO ACCESS = permission denied.

7. Analysis and Discussion

7.1 Subjects, Objects, and Access Rights

In this lab, the subjects are the three user accounts (teller2, super2, and audit2). The objects are the three directories and the files they contain. The access rights are the permission bits and ACL entries that the operating system evaluates each time a subject attempts an operation on an object.

This structure mirrors the formal access control model: a subject requests access to an object, the reference monitor (the Linux kernel) consults the security policy (permission bits and ACLs), and either permits or denies the operation. The denied operations in Part D demonstrate that the reference monitor is functioning correctly.

7.2 Relationship Between DAC and RBAC

Pure discretionary access control grants access based on user identity and ownership. Role-based access control (RBAC) grants access based on the roles a user holds, with roles serving as an intermediary between users and permissions. This lab simulates RBAC by assigning users to role groups and then granting permissions to groups rather than to individuals.

The advantage of this approach is scalability. If the bank hired ten additional tellers, each new account would simply be added to the teller group, and all appropriate permissions would be inherited immediately without any changes to the filesystem permissions. This reduces administrative error and makes the access model easier to audit.

7.3 Limitations of the Standard Permission Model

The standard chmod model assigns exactly one group to each file. When the auditor needed read access to both the transactions and approvals folders (owned by different groups), the only clean solution was ACLs. Without ACLs, the only alternatives would have been to add the auditor to multiple groups (weakening role isolation) or to relax the “others” permissions (granting access to all system accounts). Both alternatives violate the principle of least privilege.

ACLs resolve this by allowing user-specific and group-specific entries to be attached to any object, independently of the primary group. The mask entry also provides a ceiling on effective permissions, giving administrators an additional layer of control.

8. Conclusion

This laboratory exercise demonstrated that Linux discretionary access control, when combined with access control lists, provides a flexible and effective mechanism for enforcing the principle of least privilege. By mapping business roles to Unix groups and applying fine-grained ACL entries for cross-role access, it was possible to implement a permission model that accurately reflects the needs of the banking scenario: tellers can only access transaction data, supervisors can review teller records and manage approvals, and auditors can read all folders for compliance purposes without being able to modify any records.

The least-privilege tests confirmed both sides of the security guarantee. Permitted operations succeeded as expected, and prohibited operations were correctly blocked with a “Permission denied” error, proving that the reference monitor enforced the policy at the kernel level. The access matrix documents these results in a structured form that can be used as the basis for a formal access control audit.

The main security lesson from this lab is that access control must be designed at the role level, not the individual level, and that standard Unix permissions must be augmented with ACLs whenever business rules require cross-role or per-user exceptions. A well-designed access control model reduces the blast radius of a compromised account, limits insider threat opportunities, and produces a clear audit trail that satisfies compliance requirements.

Appendix – Complete Command Reference

All commands executed during this laboratory exercise are listed below in execution order for reference and reproducibility.

Environment Setup

```
mkdir -p ~/CYS121_Labs/unit2_access_control/bank_case
cd ~/CYS121_Labs/unit2_access_control/bank_case
sudo apt update
sudo apt install -y acl
date && hostname && hostname -I
```

Part A – Users, Groups, and Folders

```
sudo groupadd teller
sudo groupadd supervisor
sudo groupadd auditor
sudo adduser teller2
sudo adduser super2
sudo adduser audit2
sudo usermod -aG teller teller2
sudo usermod -aG supervisor super2
sudo usermod -aG auditor audit2
id teller2 && id super2 && id audit2
mkdir transactions approvals auditlogs
touch transactions/daybook.txt approvals/approvals.txt
auditlogs/audit.txt
echo "Daily teller summary" > transactions/daybook.txt
echo "Loan approval record" > approvals/approvals.txt
echo "System audit log entry" > auditlogs/audit.txt
```

Part B – chmod and chown

```
sudo chown -R $USER:teller transactions
chmod -R 770 transactions
sudo chown -R $USER:supervisor approvals
chmod -R 770 approvals
sudo chown -R $USER:auditor auditlogs
chmod -R 750 auditlogs
ls -ld transactions approvals auditlogs
ls -l transactions approvals auditlogs
```

Part C – ACLs

```
setfacl -m u:audit2:rx transactions
setfacl -m u:audit2:r-- transactions/daybook.txt
setfacl -m u:audit2:rx approvals
setfacl -m u:audit2:r-- approvals/approvals.txt
setfacl -m u:super2:rx transactions
setfacl -m u:super2:r-- transactions/daybook.txt
getfacl transactions approvals auditlogs
```

Part D – Least-Privilege Tests

```
sudo -u teller2 cat transactions/daybook.txt
sudo -u super2 cat approvals/approvals.txt
sudo -u audit2 cat auditlogs/audit.txt
sudo -u audit2 cat transactions/daybook.txt
sudo -u super2 cat transactions/daybook.txt
sudo -u teller2 bash -c 'echo test >> auditlogs/audit.txt'
sudo -u teller2 bash -c 'echo test >> approvals/approvals.txt'
sudo -u audit2 bash -c 'echo test >> transactions/daybook.txt'
```

THANK YOU